

Settings Reference

Every FastMagento setting, what it actually does, and how to tune it — with worked examples. The search & relevance controls are broken down clause-by-clause so you can dial in the results you want without guessing.

≡ Stores > Configuration > FastMagento

🕒 Reindex after search-attribute changes

🔒 Safe defaults · degrades to native



How to read this

Every setting is a card. The badges tell you the field type, its shipped default, and how far down the store hierarchy you can override it. Config paths are shown so you can also set them from `app/etc/config.php`, `env.php`, or `bin/magento config:set`.

Legend

Yes / No

Field type — a toggle, text box, number, dropdown, or textarea.

Default: 1

The value FastMagento ships with, out of the box.

Store-level

Can be overridden per website / store view. **Global only** = one value site-wide.

path/here

The config path — set it in the admin, or via CLI / env for automation.



Quick-start recipes

Common goals and the exact knobs that get you there. Each is explained in full below.



"Frontend UTV" shouldn't return frontend-only parts

Require more of the words to match so loosely-related items drop out.

Multi-Term Operator → **All** or **Most**



Surface products for buyer slang

Let a "UTV" part show up for "side-by-side / SxS" even if the copy never says it.

Synonyms + **AI Search Keywords**



Cut the checkout delay on configurables

Serve cart/checkout lines from the index instead of the ~217-query native path.

Enable Fast Checkout → **Yes** (default)



Rank best-sellers above the rest

Keep text relevance first, then break ties by a numeric attribute you choose.

Custom Ranking Attribute → **bestseller_score**



1 · Indexing Configuration

fastmagento/indexing/* · Names the OpenSearch indexes FastMagento reads and writes. Most stores never change these.



Real-time vs. scheduled indexing is not set here. That's controlled by Magento's own **System > Index Management** ("Update on Save" / "Update by Schedule"), because FastMagento's indexers run off Magento's native mview.

Product Index Suffix

Text

Default: products

Store-level

```
fastmagento/indexing/opensearch_index_prefix
```

The suffix for FastMagento's product index. The full index name is the engine prefix joined to this suffix.

RESULTING INDEX NAME

<catalog/search/opensearch_index_prefix>_<suffix> → e.g. magento2_products

⚠ Only change this to run parallel indexes (e.g. blue/green). After changing it, **reindex** so the new index is populated.

Category Index Suffix

Text

Default: categories

Store-level

```
fastmagento/indexing/opensearch_category_index_prefix
```

Same idea for the category index that powers menus, breadcrumbs and layered-nav attributes.

RESULTING INDEX NAME

magento2_categories

Instant Product Update on Save v2.2.0

Yes / No

Default: No

Store-level

`fastmagento/indexing/instant_product_update`

Makes a catalogue edit go live on the storefront the *moment it is saved* — instead of waiting on the scheduled indexer and full-page cache. The classic "I saved it but I still don't see it" delay, gone.

EXAMPLE

Merchandise fixes a typo in a product name and clicks Save. With the FastMagento product indexer on *Update by Schedule* (a large backlog), the fix would normally not appear until the next cron reindex. With this **On**, the OpenSearch doc is updated and the product's cached pages are purged before the save even returns — the next storefront view shows the fix.

How it works. On `catalog_product_save_commit_after` (after the DB commit, so a rolled-back save never desyncs the index), it (1) reprojects the saved product *and any configurable/grouped/bundle parent* straight into OpenSearch, bypassing the mview changelog, and (2) purges the full-page cache by dispatching Magento's own `clean_cache_by_tags` — so built-in FPC, Varnish (correctly anchored) or a CDN like Fastly each invalidate the right pages. Both halves are best-effort: a failure is logged and never blocks the save.

Off by default (opt-in): it adds a reprojection + cache purge to every product save, including admin mass actions and `productRepository->save()` loops. The native CSV importer uses direct SQL and doesn't fire the event, so bulk imports are unaffected. Adds new DI — run `bin/magento setup:di:compile` on deploy.

Category pages v2.2.0

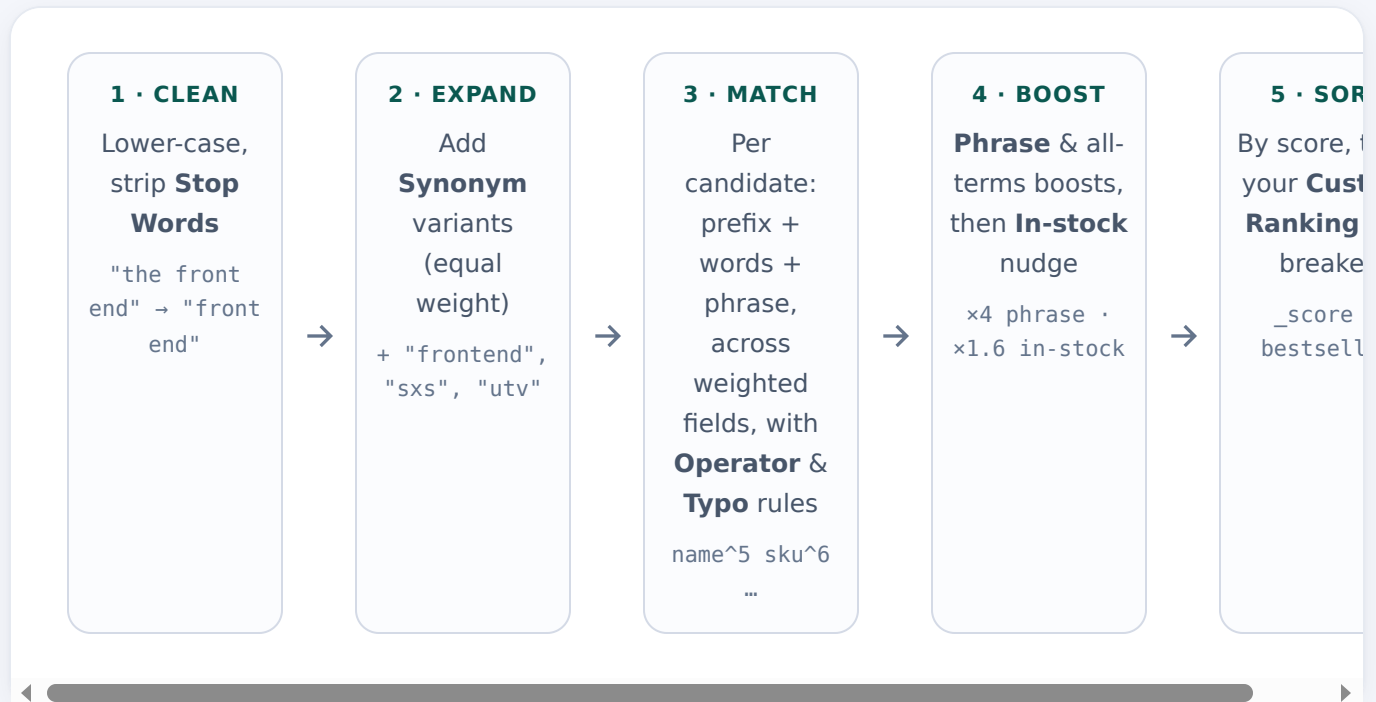
`fastmagento/indexing/instant_purge_categories` — when the cache is purged, also purge the pages of the categories the product belongs to (so listings reflect the edit, not just the product page). Root/default categories are excluded. **Default: Yes**



2 · Instant Search & Relevance

fastmagento/search/* · The knobs that decide *which* products a query returns and *in what order*. This is the section most stores need to understand — so it starts with how a query actually flows, then documents each control.

How one search query flows



Mental model: stages 1–2 decide *what counts as the query*, stage 3 decides *what matches*, and stages 4–5 decide *the order*. When results feel wrong, find the stage responsible below.

Searchable Attributes & Weights

Grid

Default: name·5, sku·6, short_description·1, description·1

Store-level

`fastmagento/search/searchable_attributes`

Which product fields are searched, and how strongly each one counts. Higher weight = a hit there ranks higher. One place to control what Magento otherwise buries in each attribute's "Search Weight".

Leave it empty to use the built-in defaults. Add a row per attribute you want searched; the weight becomes the field boost (`name^5`) in every match clause.

EXAMPLE

```
name · 5 | sku · 6 | part_type · 3 | description · 1
```

A query hitting the SKU or name outranks one that only appears in the description.

⚠ Attributes must be present in the search index. After changing this list, run `indexer:reindex catalogsearch_fulltext`.

Multi-Term Operator

Select

Default: Any

Store-level

`fastmagento/search/search_operator`

How multiple typed words combine. This is the single biggest lever on "too many vs. too few results".

Any **OR — broadest.** One matching word is enough. Most results, loosest.
(default)

Most **75% must match — balanced.** A good middle ground for 3+ word queries.

All **AND — strictest.** Every word must match. Fewest, most precise results.

WORKED EXAMPLE — QUERY "FRONTEND UTV"

Any: returns anything that says frontend or UTV — so generic frontend parts leak in.

All: returns only products matching frontend and UTV — the intended UTV front-end parts.

📘 Terms may be spread across name, keywords *and* description (cross-field) — "All" doesn't force every word into one field.

Typo Tolerance

Yes / No

Default: Yes

Store-level

`fastmagento/search/typo_tolerance`

Fuzzy matching so misspellings still find results (the OpenSearch equivalent of Algolia typo tolerance).

EXAMPLE

"contol arm" → still finds "control arm" · "suspention" → "suspension"

- Applied to the as-you-type match with `fuzziness: AUTO` (edit distance scales with word length). Turn off only if fuzzy matches are pulling in noise on very short SKUs.

Exact / Phrase Match Boost

Number

Default: 4

Store-level

`fastmagento/search/phrase_match_boost`

How hard to rank products where the query appears as a *contiguous phrase* (in name / keywords) above products where the same words are just scattered around. `0` disables it.

WORKED EXAMPLE — QUERY "ROCK SLIDER"

A product named Rock Slider Kit (exact phrase) is boosted **x4** over one whose description happens to contain "rock" and "slider" in different sentences.

Higher (6-8) Exact phrasing wins harder — good when product names are clean and descriptive.

Lower (1-2) Softer — lets strong scattered-word matches compete.

0 No phrase preference at all.

- This value also feeds the "all-terms" precision boost ($\approx 75\%$ of it), so raising it tightens phrase *and* full-coverage ranking together.

Synonyms / Thesaurus

Textarea

Pre-populated

Store-level

fastmagento/search/synonyms

One equivalence group per line, comma-separated. A search for *any* term in a group also matches the others — and they're scored *identically*, so "frontend" and "front end" return the same results in the same order.

FORMAT — ONE GROUP PER LINE

sxs, utv

a-arm, a arm, control arm, control-arm

shock, damper, coilover

 **Keep terms distinctive.** A group term that's a common catalogue word (e.g. "side" inside "side by side") over-broadens every query that expands to it, because synonym equivalents match at full strength. Put buyer *phrases* that contain common words into the **AI Search Keywords** layer instead — that's why `sxs, utv` lives here but "side by side" does not.

 Multi-word terms work: substitution happens at the phrase level, so "a-arm" $\rightleftharpoons$ "control arm" (joining/splitting words) is fine. Don't have a thesaurus yet? Generate one from your own catalogue — see [AI Assistant](#).

Stop Words

Textarea

Pre-populated

Store-level

`fastmagento/search/stopwords`

Comma- or space-separated words ignored in queries, so filler doesn't dilute the match.

DEFAULT

a, an, and, or, the, of, for, with, to, in, on, at, by, is, it, this, that, your, my

-  The raw multi-word phrase is preserved for phrase-matching even when a middle word is a stop word — so "side by side" stays intact rather than collapsing to "side side".

Facet Attributes

Text

Default: empty

Store-level

`fastmagento/search/facet_attributes`

Comma-separated attribute codes that become the filter facets on the instant-search results page. Category is always included automatically. Codes that don't exist on your catalogue are skipped.

EXAMPLE

`part_type,compatible_platforms,color,size`

-  Use **filterable, SELECT-type** attributes. Their option labels resolve cleanly from the index; multi-select attributes aren't supported here yet (they need an indexed option dictionary). **Ships empty on purpose** — a catalogue-specific default would silently break facets on stores lacking those exact codes.

Results Grid Columns

v2.1.0

Number (1-6)

Default: 4

Store-level

`fastmagento/search/grid_columns`

How many products per row the instant-search results grid shows. Set it to match your **category** pages' column count so search and category listings look consistent, instead of a hard-coded 4.

EXAMPLE

Your category pages show a 3-up grid → set this to 3 so a shopper moving from a category to a search sees the same layout.

- ① Responsive: the count is reduced automatically to at most **3 on tablet** ($\leq 1024\text{px}$) and **2 on mobile** ($\leq 640\text{px}$), so a higher desktop value never overflows a small screen.

Boost In-Stock Products

Yes / No

Default: Yes

Store-level

`fastmagento/search/boost_in_stock`

Nudges in-stock products above out-of-stock ones — *without* a hard sort, so text relevance stays primary.

- ① Implemented as a $\times 1.6$ score multiplier on in-stock items — a strongly-relevant out-of-stock product can still outrank a weakly-relevant in-stock one.

Custom Ranking Attribute & Direction

Text + Select

Default: blank · Highest first

Store-level

fastmagento/search/custom_ranking_attribute · .../custom_ranking_direction

An optional numeric attribute used as a *secondary* ranking signal, applied only after text relevance — Algolia-style custom ranking. Blank = relevance only.

EXAMPLE

custom_ranking_attribute = bestseller_score · direction = Highest first
Products tie on text relevance → the higher bestseller_score wins.

DIRECTION	MEANING	GOOD FOR
Highest first (desc)	Bigger value ranks higher	bestseller rank, rating, popularity
Lowest first (asc)	Smaller value ranks higher	price-low, "position" fields

 Products missing the attribute sort last — they're never dropped from results.

AI Search Keywords

Layer

Yes / No

Default: No

Store-level

`fastmagento/search/search_keywords_enabled`

Turns on searching a hidden, AI-generated per-product keyword field (`fm_search_keywords`). It lets a product surface for terms its own copy never uses.

EXAMPLE

A UTV part with no "side-by-side" text in its description still ranks for side-by-side / SxS because those buyer terms were generated into its keyword field.

 **Two steps to go live:** (1) populate the field off the request path with `bin/magento fastmagento:search-keywords:generate`, then (2) reindex `catalogsearch_fulltext`. Enabling the toggle alone does nothing until the field is populated.

Weight

`fastmagento/search/search_keywords_weight` — how strongly keyword hits rank (default **8**; name defaults to 5). Only used when the layer is on.

Source attrs

`.../keyword_source_attributes` — attribute codes whose labels give the generator product context (e.g. `part_type`, `make`, `model`). Blank = reuse Facet Attributes. **Global only**



3 · AI Assistant

`fastmagento/ai/*` · **Global only** Connect an AI API key to auto-build a search thesaurus and keyword layer from your own catalogue. Entirely optional — leave the key blank to keep AI features off.

API Key

Encrypted

Default: blank (off)

Global only

fastmagento/ai/claude_api_key

Stored encrypted. Used only by the generator tools below. Leave blank to disable all AI features.

Model & Max Catalogue Terms

Text

Default: model id · 1200 terms

Global only

fastmagento/ai/claude_model · fastmagento/ai/max_terms

Model — the model id to call. **Max Catalogue Terms** — an upper bound on how many attribute values + category names are sent to the model, so very large catalogues stay bounded and cost-controlled (default 1200).

Generate Thesaurus

Action

Button

Global only

Reads your attribute values + category names and builds shopper-facing synonym groups, merged into [Search > Synonyms](#).

- ✓ **Order of operations:** save the API Key first → click Generate → **review** the proposed groups → Save the Synonyms field. Nothing changes live search until you save.

CLI EQUIVALENT **V2.1.0**

Same job from the command line, for deploy pipelines: bin/magento
fastmagento:thesaurus:generate



4 · Fast Checkout

fastmagento/cart/* · Serves cart & checkout line items from OpenSearch instead of the native ~217-query product collection — the biggest single win for configurable-heavy carts.

Enable Fast Checkout

Master

Yes / No

Default: Yes (ON)

Store-level

fastmagento/cart/enable_fast_checkout

The one switch most stores use. On by default, it enables the whole fast pipeline — OS-served quote items, optimistic stock, and fast stock sync — so Fast Checkout works fully out of the box.



Cannot oversell. Order placement still re-checks salable quantity by SKU (MSI reservations); a truly out-of-stock order is rejected at placement. Any product missing/partial in the index falls back to the fully native path automatically.



After go-live, run a full reindex and place a **test order** to confirm totals. Set to **No** to fall back to the fully native cart.

Advanced toggles Granular

Yes / No x3

Default: No (implied by Master)

Store-level

These three give granular control. They're *hidden while the master is On* (which already turns them on). Each can also force *its* feature on by itself when the master is Off.

os_serve_quote_items

Serve Quote Items from OpenSearch. Hydrate cart/checkout line products from the index instead of the ~217-query collection. Hard native fallback for missing/partial, custom-option, and bundle/grouped carts.

optimistic_stock

Optimistic Stock. Skip the redundant per-load MySQL stock preload and rely on the authoritative placement-time gate. Trade-off: a briefly-stale index may only reject at the final step. *Validate with a real order.*

fast_stock_sync

Fast Stock Sync. On an order/refund/inventory save, patch only the stock fields of affected OS docs instead of reprojecting the whole product. Runs after the response — no shopper latency. Falls back to a full reproject if a doc isn't fully present.

- ① The three default to **0** by design: the master OR each flag turns a feature on, so leaving them at 0 is what lets the master switch cleanly enable *and* disable the whole bundle.

Configurable Line-Item Name

Select

Default: Parent

Store-level

`fastmagento/cart/configurable_line_name`

Which name to show for a configurable product on cart / mini-cart / checkout. Display only — it does not change price or the placed-order line data.

Parent **Configurable name** — Magento default. e.g. "Keira Banded Bra" with Color / Size beneath. *(default)*

Child **Simple (variant) name** — the specific purchased variant's own name.



5 · Cache Warmup

`fastmagento/cache/*`

Popular Search Terms

Textarea

Default: blank

Store-level

`fastmagento/cache/popular_searches`

A comma-separated list of your highest-traffic search terms. The scheduled cache-warmup cron pre-renders these result pages so the first real shopper of the day hits a warm cache instead of a cold render.

EXAMPLE

a-arm, rock slider, light bar, rZR cage, tie rod

 Warmed by the `fastmagento_cache_warmup` cron. Keep the list short and genuinely popular — warming rarely-used terms just wastes cache.



6 · Extension Efficiency Monitor

fastmagento/efficiency/* · **Global only** Profiles how much database work each third-party extension adds to the hot paths (indexing, product pages, category grids, search, logged-in customer data). View it under **FastMagento > Extension Efficiency**.

Run Scan on a Schedule

Yes / No

Default: No

Global only

fastmagento/efficiency/cron_enabled · .../cron_expr

Off by default. When on, a scan runs automatically on the schedule and refreshes the monitor. The schedule is a standard cron expression (default `3 * * 0` = Sundays 3:00 AM).



A scan briefly enables query logging to attribute SQL to modules — so it's opt-in. **Don't overlap a full reindex.** You can always run one on demand from the admin page or with `bin/magento fastmagento:efficiency:scan`.

Sample Size

Number

Default: 50

Global only

fastmagento/efficiency/sample_size

Products / rows profiled per scenario. Higher = steadier averages but a slower scan.



7 · Footer Credit

fastmagento/credit/* · A small, honest attribution link in the storefront footer.

Show "Powered by FastMagento" credit

Yes / No

Default: Yes

Store-level

`fastmagento/credit/show`

Renders a subtle, class-based "Powered by FastMagento" credit in the footer — a real, visible attribution link (never hidden or cloaked). It's the one thing the project asks in return, and it helps other merchants find the extension.

- ① If it doesn't fit your footer, please **restyle it rather than removing it** — it's plain and class-based, so a couple of lines of CSS reskin it (`.fm-credit` , `.fm-credit__link`), or override the template `ParkkTech_FastMagento::footer-credit.phtml` in your theme.



8 · Advanced (config-only)

These have no admin screen — set them via `app/etc/config.php` , `env.php` , or `bin/magento config:set` . Defaults are correct for almost everyone.

Attribute-Option Pagination

Flag + Number

Default: On · 50 per page

Store-level

`fastmagento/attribute_pagination/enabled` · `.../page_size`

Replaces Magento's native "Manage Options" / swatch grid — which loads and re-saves the *entire* option set and buckles past a few thousand options — with a paginated, per-row CRUD manager.

- ① Leave On unless you specifically need the native grid. Only matters for attributes with very large option sets (e.g. thousands of colors/sizes).

Attribute-Option Index Suffix

Text

Default: attribute_options

Store-level

fastmagento/indexing/opensearch_option_index_prefix

Suffix for the attribute-option index (→ `magento2_attribute_options`). Same convention as the product/category suffixes; rarely changed.



Paths & CLI cheat-sheet

Set any value from the CLI (great for CI / multi-store)

```
bin/magento config:set fastmagento/search/search_operator all
bin/magento config:set --scope=stores --scope-code=default
fastmagento/cart/enable_fast_checkout 1
```

After changing searchable attributes / AI keywords

```
bin/magento fastmagento:search-keywords:generate # optional: build the AI keyword
layer
bin/magento fastmagento:thesaurus:generate # optional: AI synonym groups (CLI parity
with the admin button)
bin/magento indexer:reindex catalogsearch_fulltext fastmagento_product
fastmagento_category
```

Turn on instant product update (edits go live on save)

```
bin/magento config:set fastmagento/indexing/instant_product_update 1
bin/magento setup:di:compile # new DI wiring; production mode
```

Efficiency monitor on demand

```
bin/magento fastmagento:efficiency:scan --sample=50
```

Flush after config changes

```
bin/magento cache:flush
```

FastMagento — Settings Reference

Admin location: **Stores > Configuration > FastMagento Settings**. Config paths shown on each card also work via `bin/magento config:set`, `app/etc/config.php` and `env.php`. All features degrade safely to native Magento on any miss or outage.